



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

Backtracking 1 - Recursion การย้อนรอย 1 - การเวียนเกิด

ผศ. ดร.สรวิ รัตนสัญญา

san.r@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Topics

- ความสัมพันธ์เวียนเกิด (Recurrence Relations)
- การเวียนเกิด (Recursion)
 - การเวียนเกิดคืออะไร?
 - ประเภทของการเวียนเกิด
 - ลักษณะของปัญหาที่เหมาะสมกับการใช้งาน
 - ข้อดี-ข้อเสีย



ความสัมพันธ์เวียนเกิด (Recurrence relation)

- หรือ Recurrence Relations สำหรับอันดับ $a_0, a_1, \dots, a_n$ เป็นสมการที่แสดงความสัมพันธ์ของพจน์ a_n กับพจน์ $a_0, a_1, \dots, a_{n-1}$ ที่เกิดก่อน โดยมีเงื่อนไขเริ่มต้นสำหรับพจน์ $a_0, a_1, \dots, a_{n-1}$ ที่ชัดเจน หรืออาจจะกล่าวได้ คือตัวเลขถัดไปนั้นมีความสัมพันธ์กับตัวเลขก่อนหน้า โดยสามารถนำมาเขียนสมการคณิตศาสตร์ได้

- ตัวอย่าง จงหาความสัมพันธ์เวียนเกิดของอันดับ 5, 8, 11, 14, 17,...

$$a_0 = 5$$

$$a_1 = 8 = 5+3 = a_0+3$$

$$a_2 = 11 = 8+3 = a_1+3$$

$$a_3 = 14 = 11+3 = a_2+3$$

$$a_4 = 17 = 14+3 = a_3+3$$

พิจารณาพบว่า พจน์ถัดไปมีค่า
มากกว่าพจน์ก่อนหน้าอยู่ 3

พิจารณาต่อไปจะได้

$a_n = a_{n-1}+3 \ ; \ n \geq 1$	เป็น ความสัมพันธ์เวียนเกิด
โดยมี $a_0 = 5$	เป็น เงื่อนไขเริ่มต้น

- **แฟกทอเรียล** ของจำนวนเต็มไม่ติดลบ n คือ ผลคูณของจำนวนเต็มบวกทั้งหมดที่น้อยกว่าหรือเท่ากับ n เขียนแทนด้วย $n!$
- คำตอบเกิดจากการคูณของจำนวนเต็มบวกชุดหนึ่ง ซึ่งถ้าคำตอบเกิดจากการคูณของจำนวนเต็มบวกตั้งแต่ 1 ถึง n เช่น $1 \times 2 \times 3 \times 4 \times 5$ จำนวนเหล่านี้เราสามารถใช้สัญลักษณ์ แฟกทอเรียล เขียนแทนได้ คือ $5!$

$$\begin{aligned} N! &= N \times (N-1)! \\ &= N \times (N-1) \times (N-2)! \\ &= N \times (N-1) \times (N-2) \times (N-3) \times \dots \times 3 \times 2 \times 1 \end{aligned}$$

โดย $0!$ มีค่า เป็น 1

โดย $1!$ มีค่า เป็น 1

แฟกทอเรียล

- จากนิยามสามารถสรุปการหาคำตอบของ $N!$ ได้เป็น 2 กรณี คือ
 - ถ้า N มีค่าเท่ากับ 0 คำตอบที่ได้คือ $N! = 1$
 - ถ้า N มีค่ามากกว่า 0 คำตอบที่ได้คือ $N! = N \times (N-1)!$
- สำหรับ $5!$ การคำนวณหาค่าแฟกทอเรียล

$$\begin{aligned}
 5! &= 5 \times 4! \\
 &= 5 \times 4 \times 3! \\
 &= 5 \times 4 \times 3 \times 2! \\
 &= 5 \times 4 \times 3 \times 2 \times 1! \\
 &= 5 \times 4 \times 3 \times 2 \times 1 = 120
 \end{aligned}$$

1	ft = 5 * fact(4);	
2	ft = 5 * 4 * fact(3);	
3	ft = 5 * 4 * 3 * fact(2);	
4	ft = 5 * 4 * 3 * 2 * fact(1);	
5	ft = 5 * 4 * 3 * 2 * 1;	หยุดการทำงาน

แฟกทอเรียล

$$n! = 1 \times 2 \times 3 \dots \times (n-1) \times n = (n-1)! \times n$$

$$a_n = (n-1)! \times n$$

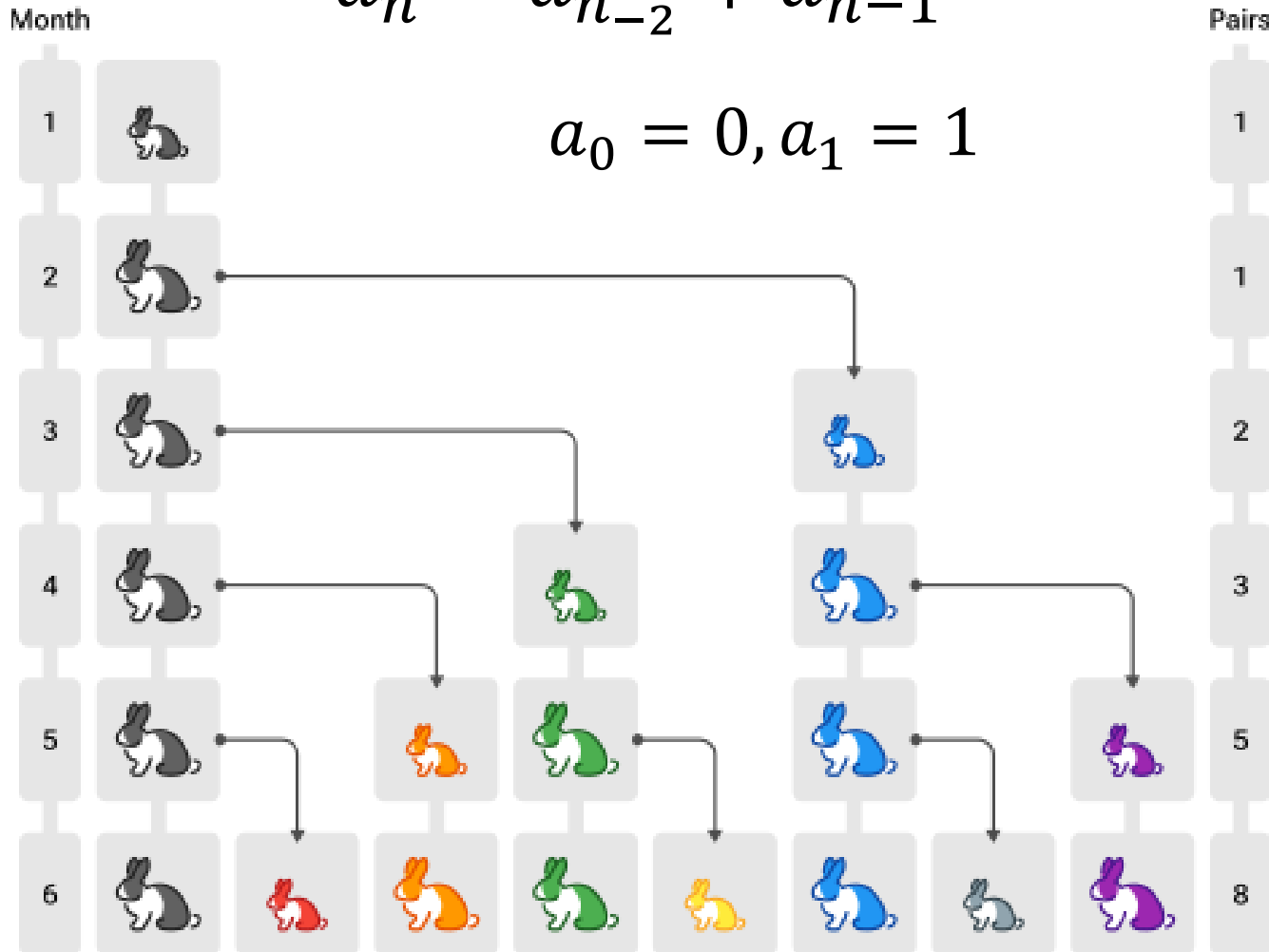
$$= a_{n-1} \times n$$

- ดังนั้น ความสัมพันธ์เวียนเกิดของ $n!$
- คือ $a_n = n a_{n-1} ; n \geq 1$
- เงื่อนไขเริ่มต้น $a_0 = 1$ และ $a_1 = 1$

Fibonacci

$$a_n = a_{n-2} + a_{n-1}$$

$$a_0 = 0, a_1 = 1$$



Golden Ratio



- ลำดับฟีโบนัชชี (Fibonacci Sequence) มีนิยามของความสัมพันธ์ว่า จำนวนถัดไปเท่ากับผลบวกของจำนวนสองจำนวนก่อนหน้า และสองจำนวนแรกก็คือ 0 และ 1 ตามลำดับ หากเขียนให้อยู่ในรูปของสัญลักษณ์ ลำดับ F_n ของฟีโบนัชชี
- สามารถเขียนความสัมพันธ์เวียนเกิดได้ดังนี้

$$F_n = F_{n-1} + F_{n-2}$$

โดยกำหนดค่าเริ่มแรกให้

$$F_0 = 0 \text{ และ } F_1 = 1$$

ฟีโบนัชชี

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 ,... จากตัวเลขอนุกรมดังกล่าว
สามารถแสดงวิธีการหาค่าเทอมต่างๆ ได้ดังนี้

1) $F_0 = 0$

2) $F_1 = 1$

3) $F_n = F_{n-1} + F_{n-2}$

พจน์ที่ 3 มาจาก พจน์ที่ 1 + พจน์ที่ 2 = $0 + 1 = 1$ โดยเริ่มจากพจน์ที่ 3 เป็นต้นไป

พจน์ที่ 4 มาจาก พจน์ที่ 2 + พจน์ที่ 3 = $1 + 1 = 2$

พจน์ที่ 5 มาจาก พจน์ที่ 3 + พจน์ที่ 4 = $1 + 2 = 3$

ดังนั้น ความสัมพันธ์เวียนเกิด คือ $a_n = a_{n-1} + a_{n-2} ; n \geq 2$

และเงื่อนไขเริ่มต้น $a_0 = 0$ และ $a_1 = 1$

จากนิยามสามารถสรุปการหาคำตอบออกเป็น 2 ทางคือ

ถ้า n มีค่าเป็น 0 หรือ 1 คำตอบที่ได้คือ $F_n = n$

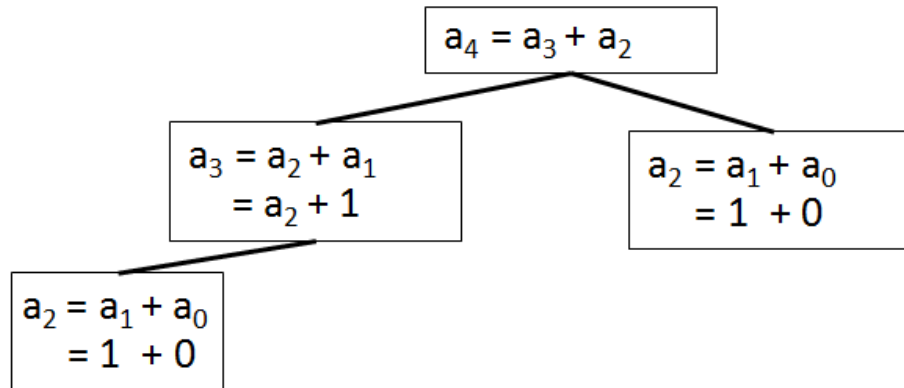
ถ้า n มีค่ามากกว่า 1 คำตอบที่ได้คือ $F_n = F_{n-1} + F_{n-2}$

ฟีโบนัชชี

- จงคำนวณหาค่าอนุกรมฟีโบนัชชีที่ 4
- วิธีคิดแบบต้นไม้แตกกิ่งก้านสาขา

- วิธีคิดแบบปกติ

$$\begin{aligned}
 F(4) &= F(3) + F(2) \\
 &= (F(2) + F(1)) + (F(1) + F(0)) \\
 &= ((F(1) + F(0)) + F(1)) + (F(1) + F(0)) \\
 &= ((1+0) + 1) + (1 + 0) \\
 &= 3
 \end{aligned}$$



ตัวอย่าง 2

- มีเงินฝาก 1,000 บาท อัตราดอกเบี้ย 12% ต่อปี ถ้าฝากแบบดอกเบี้ยทบต้นสิ้นปีที่ n จะมีเงินรวมทั้งหมดเท่าไร

สิ้นปีที่ 1 มีเงิน $1000.00 + (0.12)(1000.00) = 1120.00$

สิ้นปีที่ 2 มีเงิน $1120.00 + (0.12)(1120.00) = 1254.40$

สิ้นปีที่ 3 มีเงิน $1254.40 + (0.12)(1254.40) = 1404.92$

...

สิ้นปีที่ n มีเงิน เงินรวมสิ้นปีที่ $n-1$ + ดอกเบี้ย

ให้ a_n เป็นเงินรวมเมื่อสิ้นปีที่ n

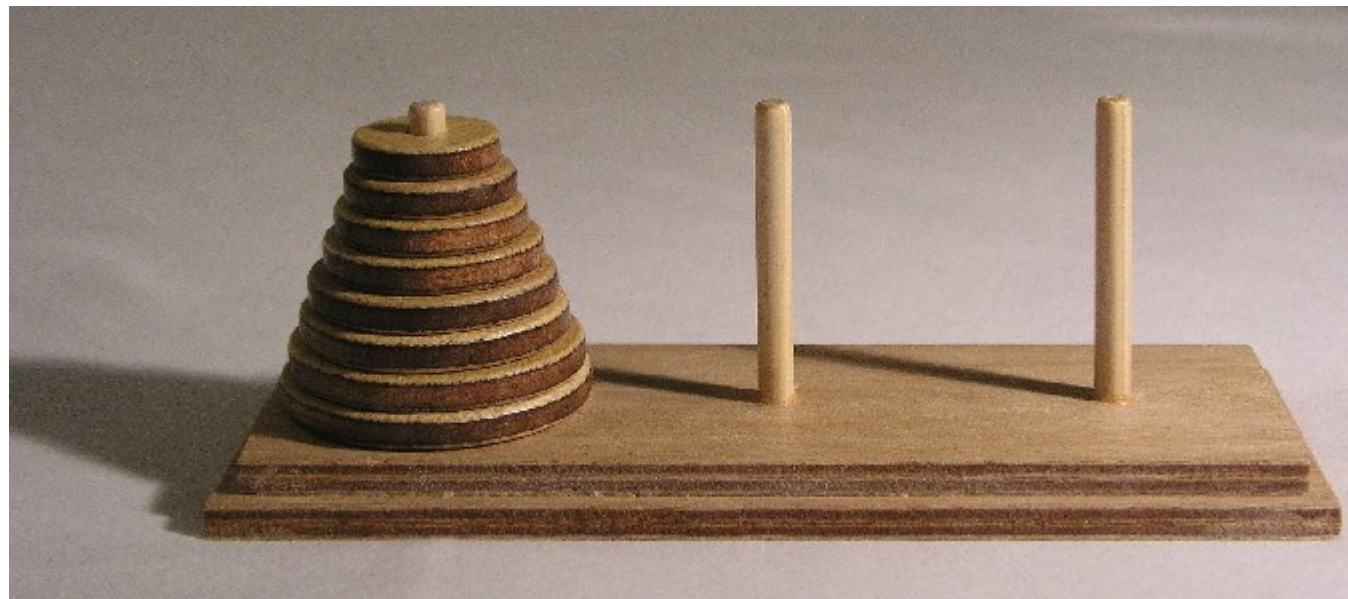
$$a_n = (1)(a_{n-1}) + (0.12)(a_{n-1})$$

$$a_n = (1.12)(a_{n-1})$$

- ดังนั้น ความสัมพันธ์เวียนเกิด คือ $a_n = (1.12)(a_{n-1}) ; n \geq 1$
และเงื่อนไขเริ่มต้น $a_0 = 1000$

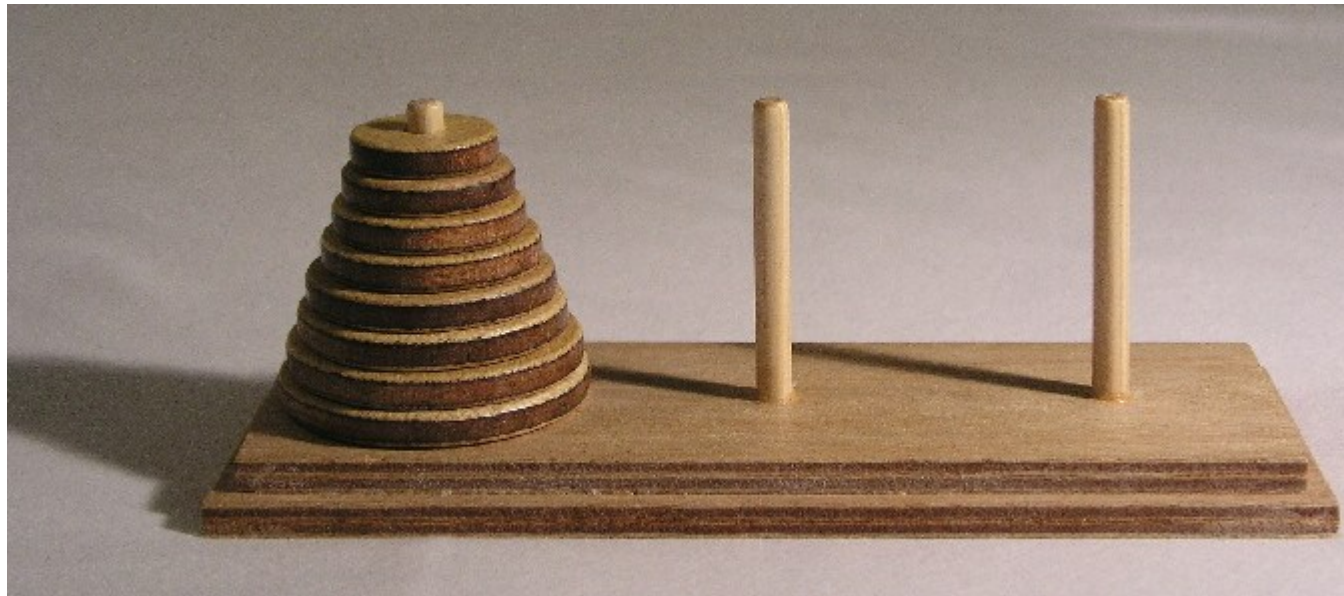
ปริศนาหอคอยฮานอย

- EDOUARD LUCAS คือ นักคณิตศาสตร์ชาวฝรั่งเศส เป็นผู้คิดค้นปริศนาหอคอยฮานอย (The Tower of Hanoi) โดยปริศนาหอคอยฮานอย นั้นจะมีแผ่นจานไม้ 8 แผ่น รัศมีแตกต่างกัน แต่ละแผ่นมีรูตรงกลาง นำมาใส่ไว้ในหลักเป็นกองซ้อน โดยให้แผ่นที่เล็กกว่าทับแผ่นที่ใหญ่กว่า และมีหลักเปล่าสองหลัก ดังรูป

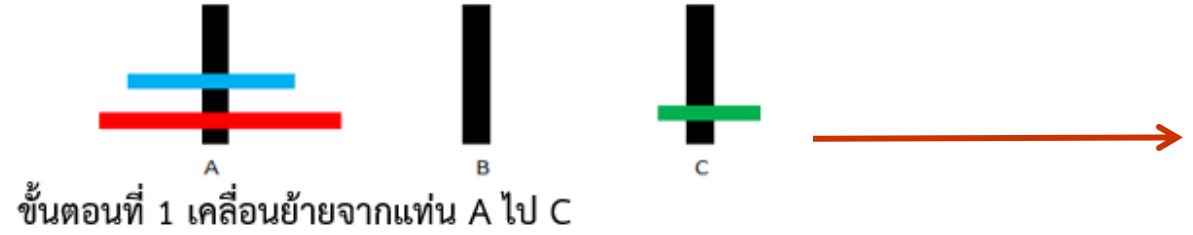


ปริศนาหอคอยฮานอย

- **ปริศนาหอคอยฮานอย** คือ ให้ย้ายแผ่นจานทั้งหมดไปกองไว้ที่หลักเปล่าหลักหนึ่ง โดยมีเงื่อนไขว่า เคลื่อนย้ายได้คราวละแผ่น และต้องนำไปไว้ที่หลักใดหลักหนึ่ง และห้ามแผ่นที่มีขนาดใหญ่กว่าวางทับแผ่นที่มีขนาดเล็กกว่า ต่อไปนี้แสดงขั้นตอนการเคลื่อนย้ายแผ่นจานจำนวน 3 แผ่นจากแผ่น A ไปแผ่น C

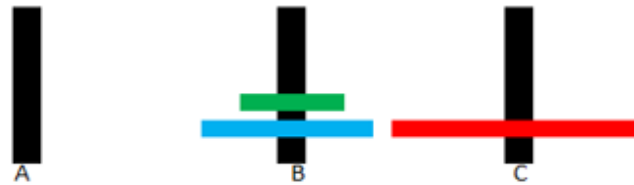


ตัวอย่างการเคลื่อนย้าย



ขั้นตอนที่ 2 เคลื่อนย้ายจากแท่น A ไป C

ขั้นตอนที่ 3 เคลื่อนย้ายจากแท่น C ไป B



ขั้นตอนที่ 4 เคลื่อนย้ายจากแท่น A ไป C

ขั้นตอนที่ 5 เคลื่อนย้ายจากแท่น B ไป A



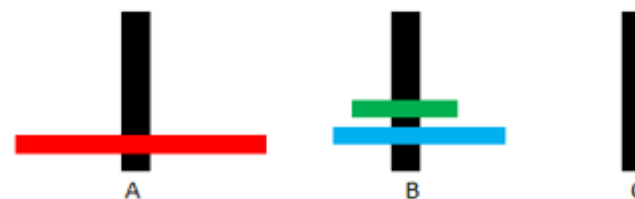
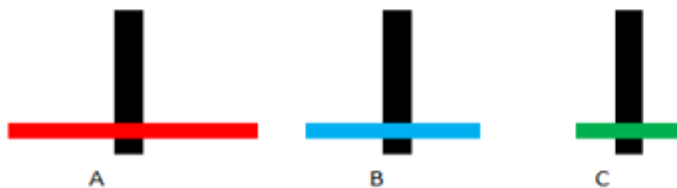
ขั้นตอนที่ 6 เคลื่อนย้ายจากแท่น B ไป C

ขั้นตอนที่ 7 เคลื่อนย้ายจากแท่น A ไป C

ตัวอย่างการเคลื่อนย้าย

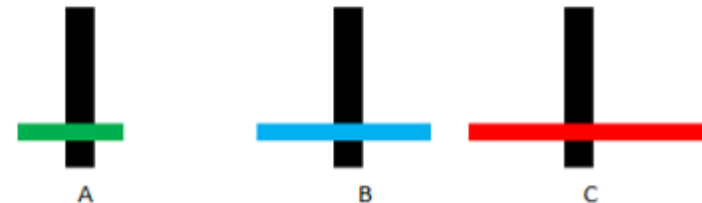


ขั้นตอนที่ 1 เคลื่อนย้ายจากแท่น A ไป C



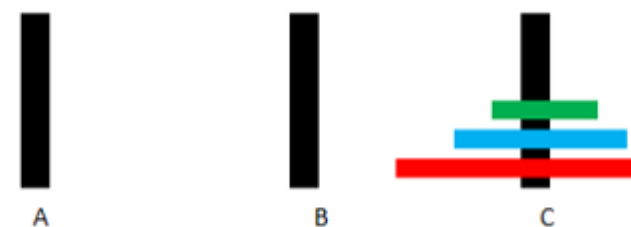
ขั้นตอนที่ 3 เคลื่อนย้ายจากแท่น C ไป B

ขั้นตอนที่ 2 เคลื่อนย้ายจากแท่น A ไป C



ขั้นตอนที่ 5 เคลื่อนย้ายจากแท่น B ไป A

ขั้นตอนที่ 4 เคลื่อนย้ายจากแท่น A ไป C

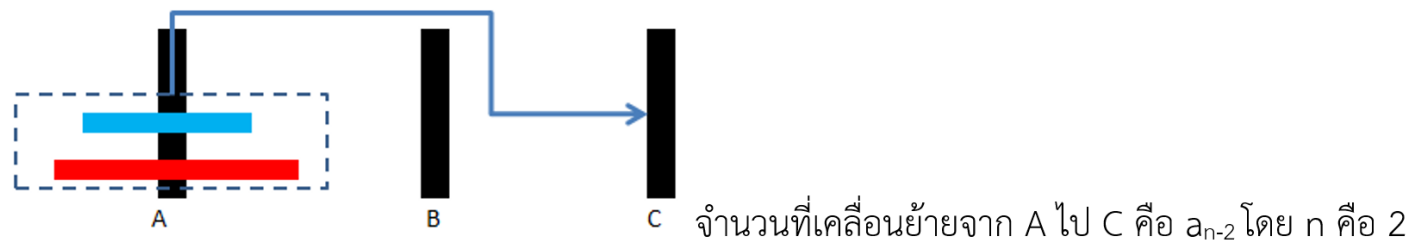
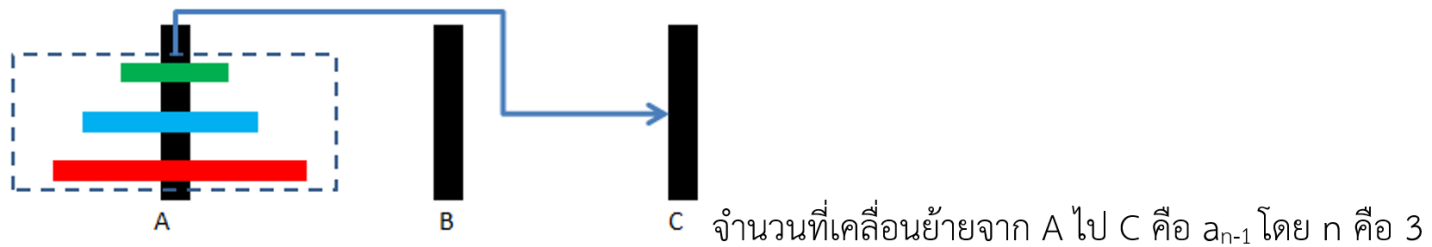
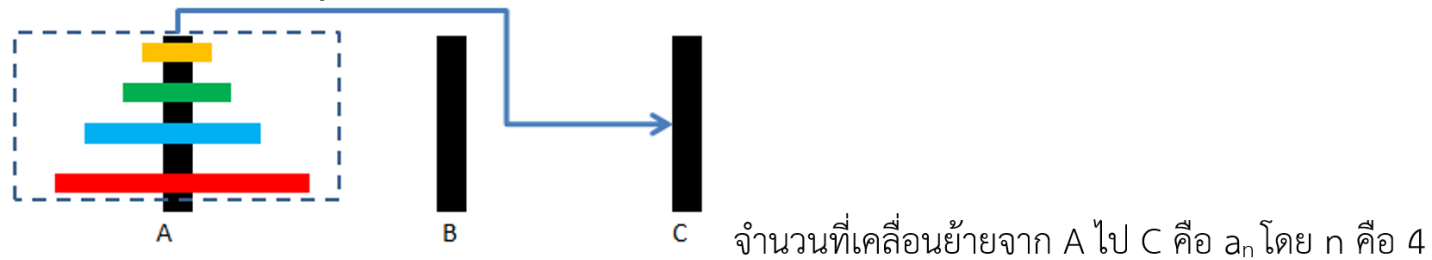


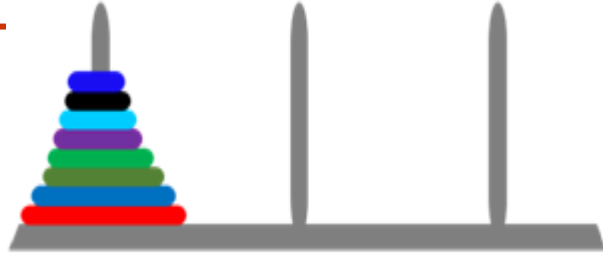
ขั้นตอนที่ 7 เคลื่อนย้ายจากแท่น A ไป C

ขั้นตอนที่ 6 เคลื่อนย้ายจากแท่น B ไป C

จำนวนการเคลื่อนย้ายแผ่น

- ให้ a_n เป็นจำนวนครั้งน้อยสุดในการเคลื่อนย้ายแผ่นจากหลักหนึ่งไปยังอีกหลักหนึ่ง





1. ต้องย้ายแผ่นที่เล็กกว่าทั้งหมด $n-1$ แผ่น
ไปยังหลักที่ว่างก่อน จำนวนการเคลื่อนย้าย
คือ a_{n-1} ครั้ง



2. ย้ายแผ่นใหญ่ที่สุดไปยังหลักเป้าหมาย
จำนวนการเคลื่อนย้ายคือ 1 ครั้ง



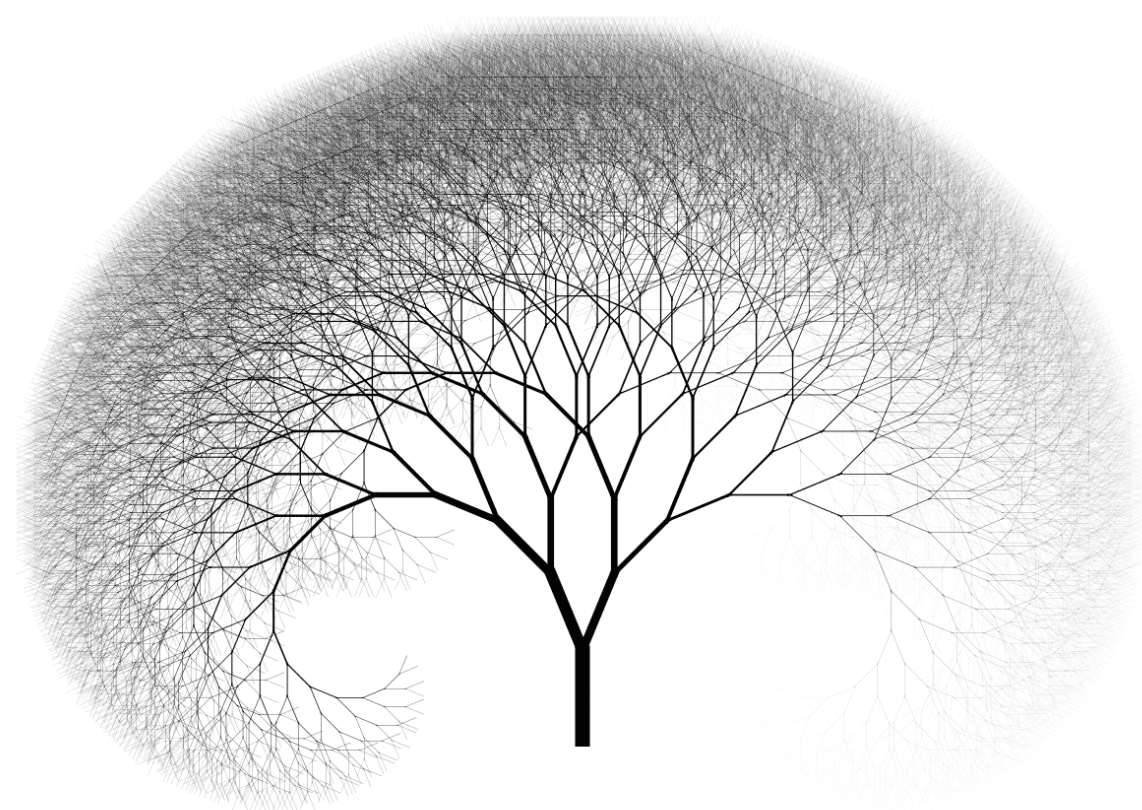
3. ย้ายแผ่นจานทั้งหมดในข้อ 1. ไปยัง
หลักเป้าหมาย จำนวนการเคลื่อนย้าย
คือ a_{n-1} ครั้ง



จะได้ $a_n = a_{n-1} + 1 + a_{n-1}$
 $= 2a_{n-1} + 1$

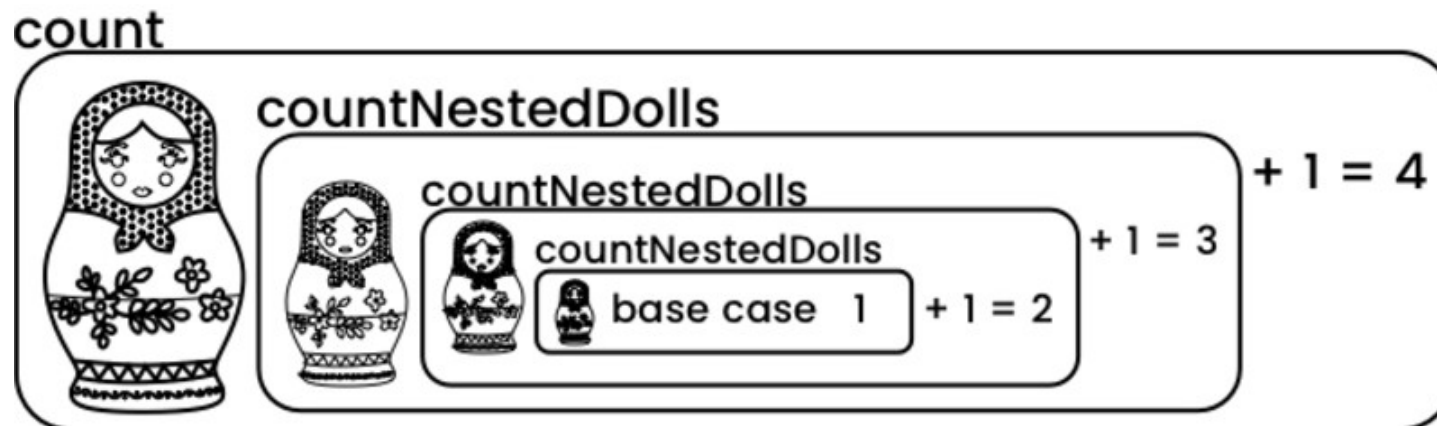
การเวียนเกิดคืออะไร?

- เป็นวิธีการเขียนโปรแกรม หรือ ฟังก์ชันแบบหนึ่ง
- ฟังก์ชันที่มีการใช้งานเรียกตัวเอง
 - ฟังก์ชันเวียนเกิด (Recursive function)
- คล้ายการวนซ้ำ (Loop)



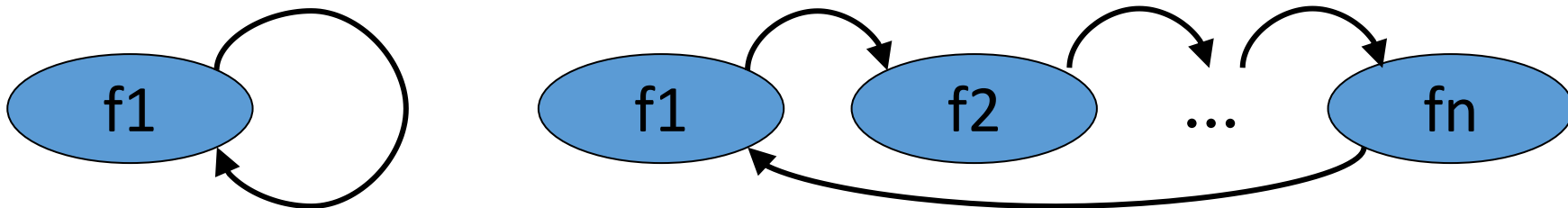
ฟังก์ชันเวียนเกิด (Recursive function)

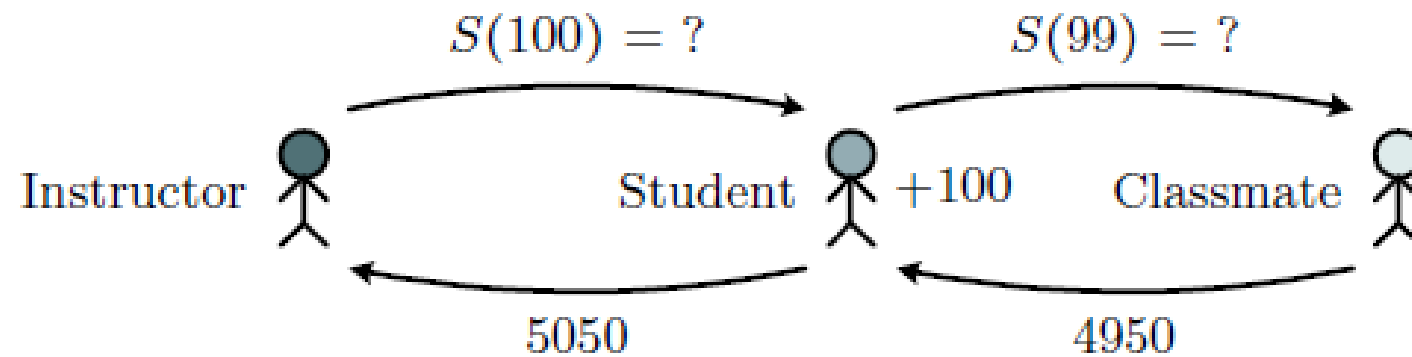
- ฟังก์ชันที่เรียกตัวเองซ้ำไปซ้ำมาจนกว่าจะแก้ปัญหาได้
- แนวคิดในการแก้ปัญหา ด้วยการแบ่งปัญหานั้นออกเป็นปัญหาย่อย ๆ โดยที่
 - ปัญหาย่อยเหล่านั้นมีโครงสร้างเดียวกันกับปัญหาดั้งเดิม ซึ่งทำให้สามารถแก้ปัญหาได้ง่ายกว่า
 - เมื่อนำผลลัพธ์ของปัญหาย่อยเหล่านั้นมารวมกัน จะได้คำตอบของปัญหาดั้งเดิม
- อยู่บนพื้นฐานของการอุปนัย หรือการเหนี่ยวนำทางคณิตศาสตร์ (Mathematical Induction)



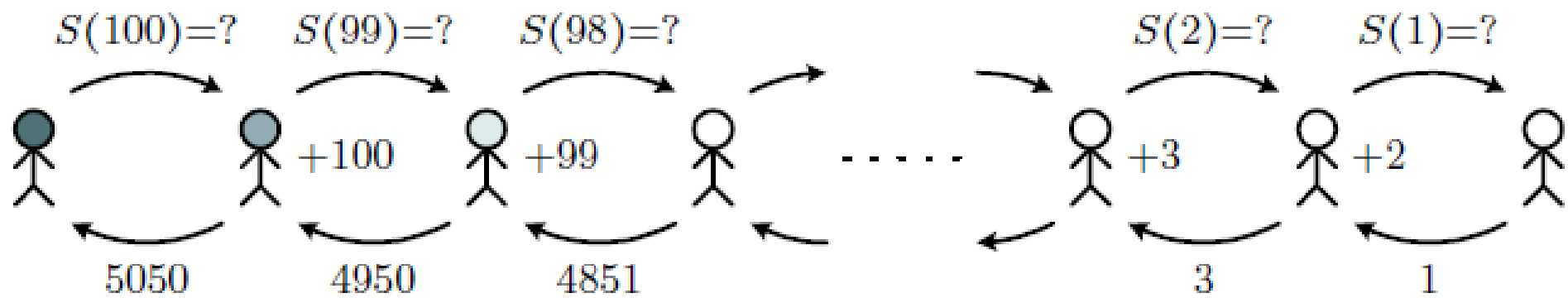
ฟังก์ชันเวียนเกิด

- ฟังก์ชันที่เป็นส่วนหนึ่งของวงรอบในลำดับการเรียกใช้ฟังก์ชัน





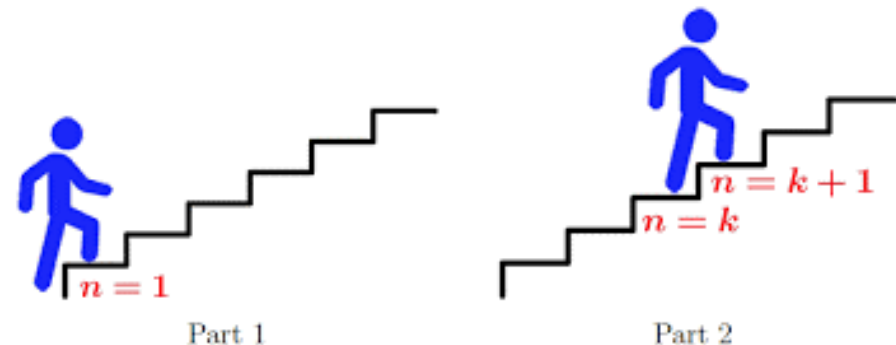
(a)

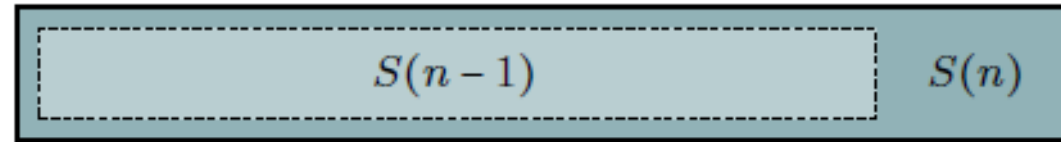


(b)

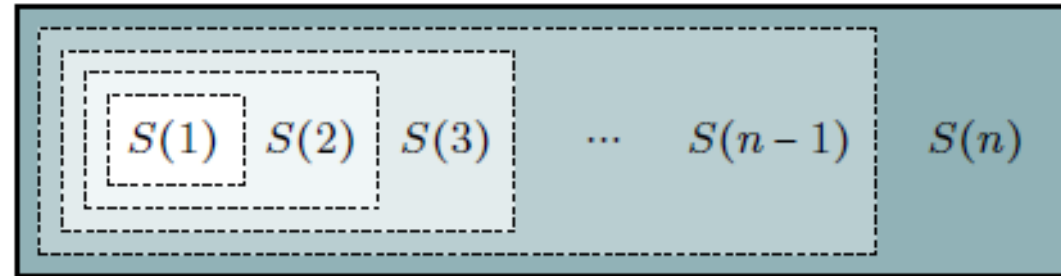
การเหนี่ยวนำทางคณิตศาสตร์ (Mathematical Induction)

- การอุปนัย หรือ การเหนี่ยวนำทางคณิตศาสตร์ ใช้กับจำนวนเต็มที่มีมากกว่า 0
- Base case: ให้ $P(1)$ เป็นจริง
- Inductive case (Recursive case): ให้ $P(2) \dots P(k)$ เป็นจริง พิสูจน์ว่า $P(k+1)$ เป็นจริง



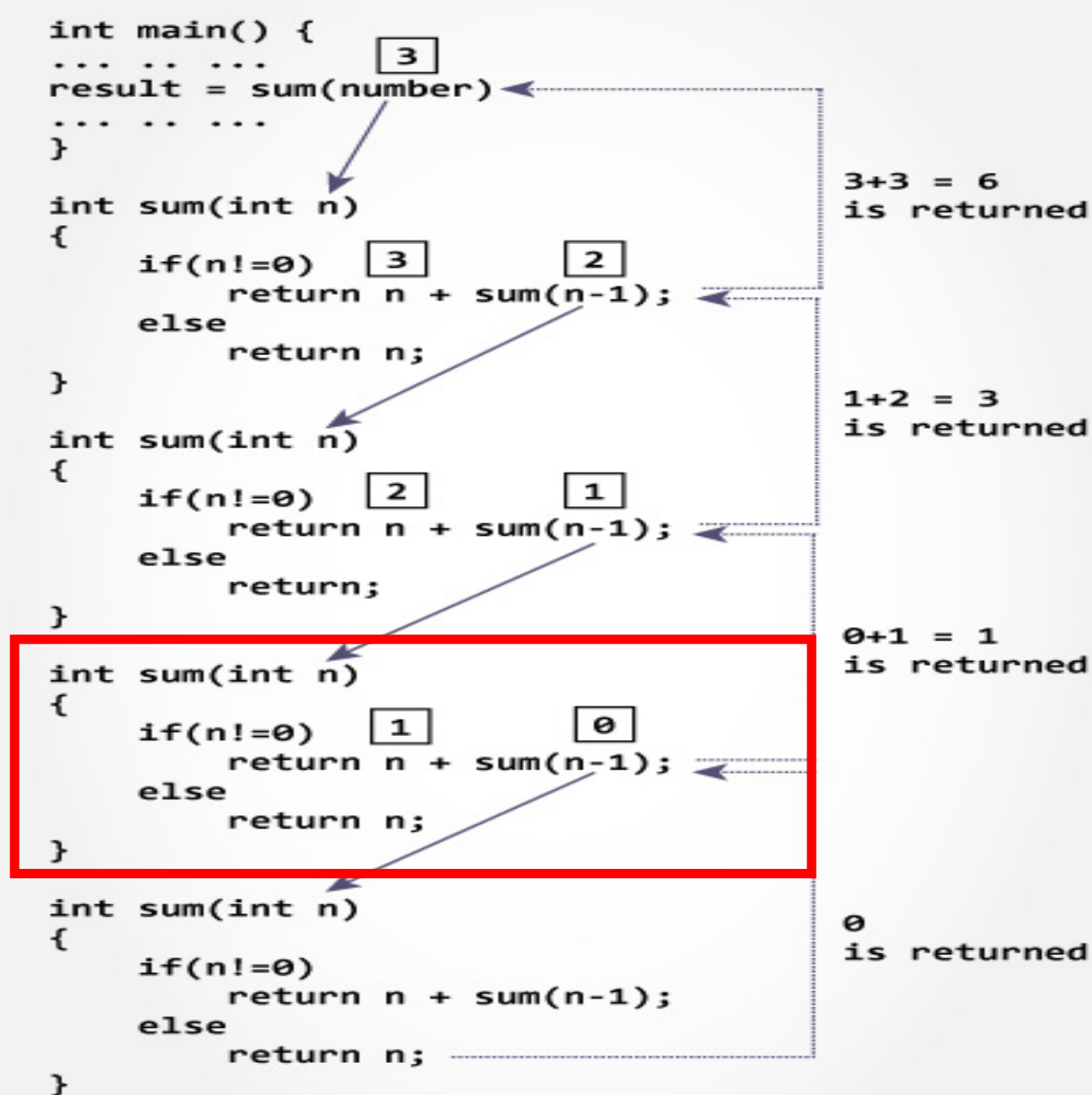


(a)



(b)





สรุปการทำงานของฟังก์ชันเวียนบังเกิด

- ฟังก์ชันหนึ่ง**เรียกใช้งานตัวเอง** โดยแต่ละครั้งที่ตัวมันเองถูกเรียก
- ปัญหาจะถูกแบ่งเป็น**ปัญหาย่อย (Recursive case)** ที่มี**ขนาดเล็กลง**
- และจะเกิดค่าตัวแปรหรือพารามิเตอร์**เปลี่ยนแปลง**ไปอย่างอัตโนมัติ
- แล้วกำหนด**การทำงานขั้นสุดท้าย (Base case)** ไว้ เมื่อทำงานถึงขั้นสุดท้ายก็จะสิ้นสุดการทำงานและส่งผลลัพธ์กลับไป

- เมื่อมีกรณีอย่างน้อย 1 กรณีที่สามารถหาคำตอบได้ทันที (base case หรือ simple case)
- ในขณะที่กรณีอื่นๆสามารถกำหนดได้ใหม่ (redefine) ในรูปที่ง่ายขึ้นกว่าเดิม เรียกว่า recursive case
- ปัญหาสามารถลดรูปลงมาสู่ simple case
 - *If this is a simple case*
solve it
 - else*
redefine the problem using recursion

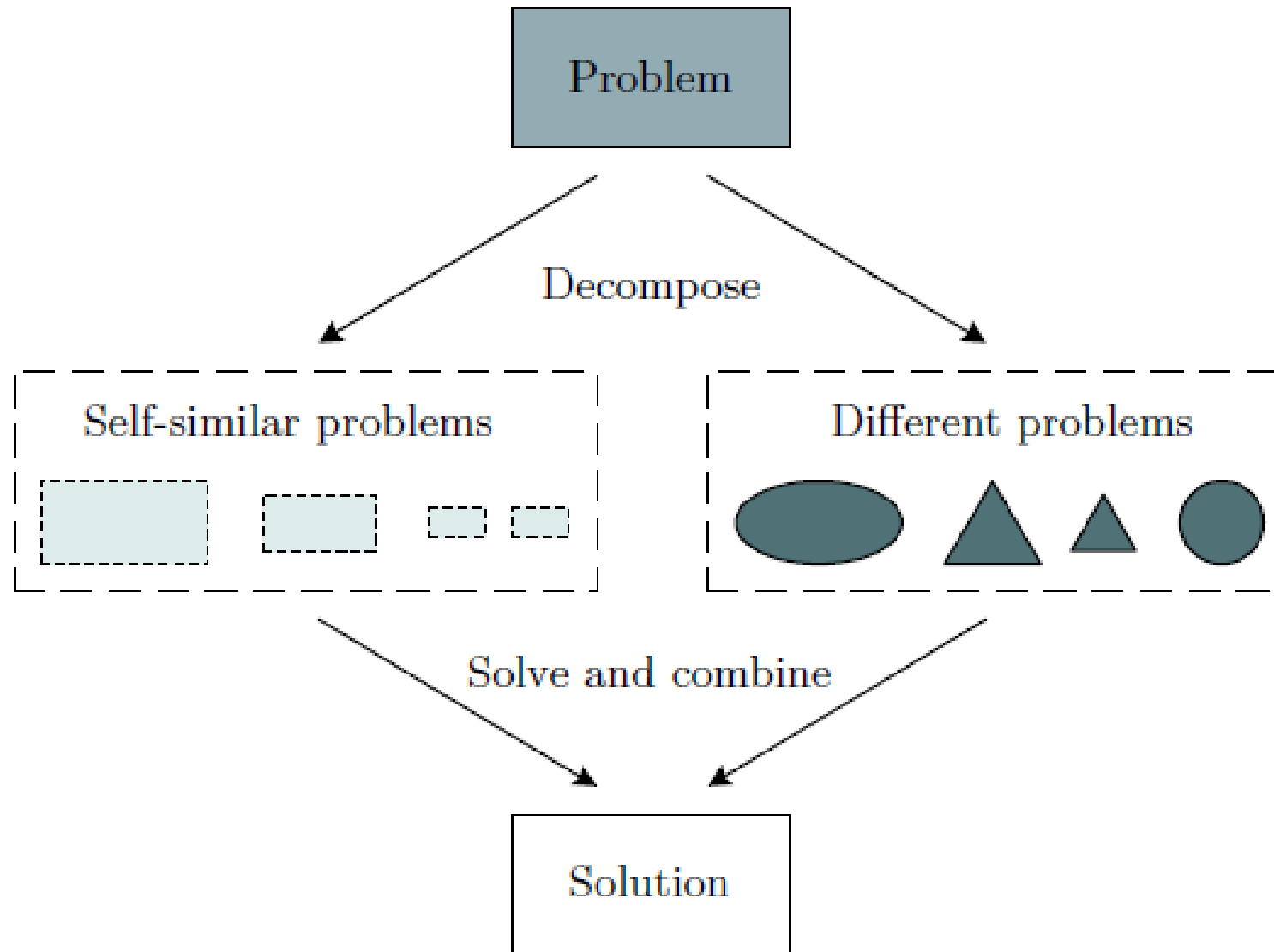
เงื่อนไขในการหยุดการเวียนบังเกิด (Terminating condition)

- recursive functions จะต้องมียุเงื่อนไขหยุดการทำงานอย่างน้อย 1 เงื่อนไขเสมอ
 - โดยปกติ จะเป็นเงื่อนไขของกรณีพื้นฐาน base case
- หากไม่มี terminating condition ฟังก์ชันอาจทำงานไม่หยุดเลย
 - เช่น ฟังก์ชันการคูณ if statement “if (n == 1) ...” คือ terminating condition.



การออกแบบฟังก์ชันเวียนบังเกิด (Designing recursive function)

- 1) หาขนาดของปัญหา (n)
- 2) กำหนด Base case
- 3) แบ่งปัญหออกเป็นปัญหาย่อยที่มีโครงสร้างเหมือนปัญหาดั้งเดิม
- 4) กำหนด Recursive case ด้วยหลักการอุปนัย
- 5) ทดสอบโปรแกรม



ตัวอย่าง (Example)

```
#include <stdio.h>
int sum(int n);

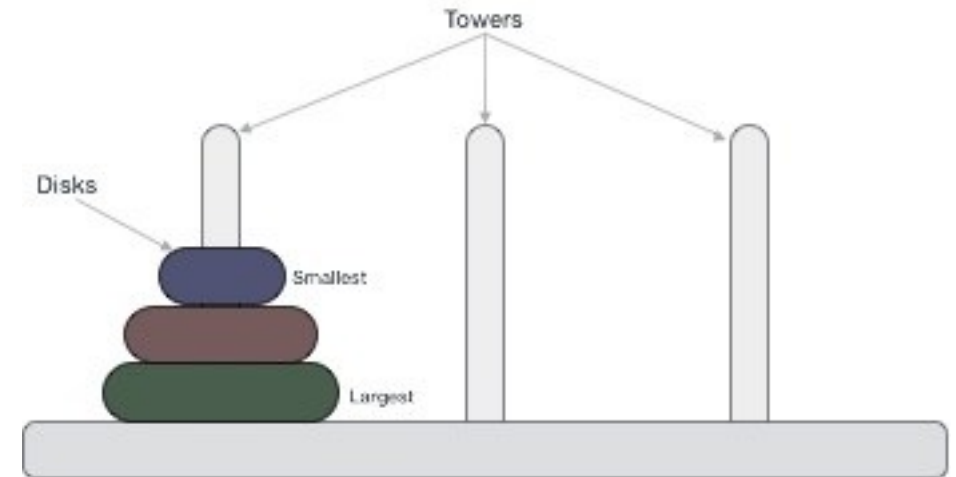
int main() {
    int number, result;

    printf("Enter a positive integer: ");
    scanf("%d", &number);
    result = sum(number);
    printf("sum = %d", result);
    return 0;
}
```

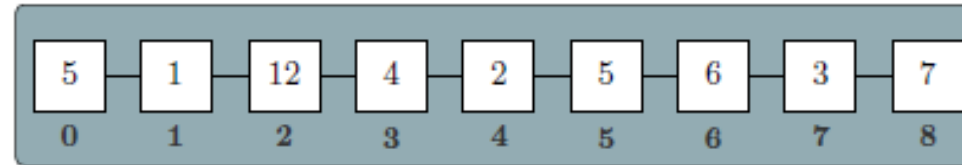
```
int sum(int num) {
    if (num!=0)
        return num + sum(num-1); // sum() function calls itself
    else
        return num;
}
```

รูปแบบของการเวียนเกิด (Types of recursion)

- Linear
 - Basic
 - Tail
- Multiple
 - Divine and Conquer
 - Puzzles and Fractal

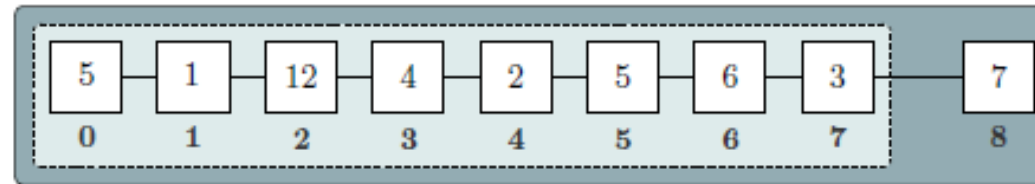


Original problem: $s(\mathbf{a}) = \mathbf{a}[0] + \mathbf{a}[1] + \dots + \mathbf{a}[n-1]$



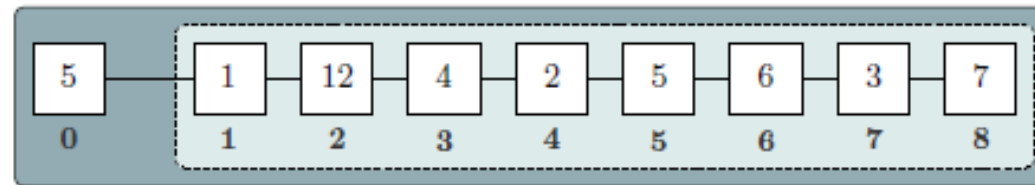
(a)

$$s(\mathbf{a}) = s(\mathbf{a}[0:n-1]) + \mathbf{a}[n-1]$$



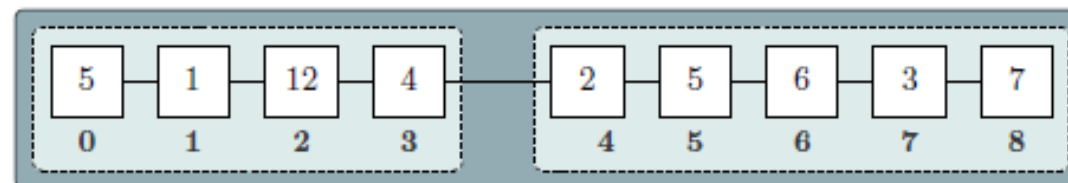
(b)

$$s(\mathbf{a}) = \mathbf{a}[0] + s(\mathbf{a}[1:n])$$



(c)

$$s(\mathbf{a}) = s(\mathbf{a}[0:n/2]) + s(\mathbf{a}[n/2:n])$$



(d)



How does recursion work?

```
void recurse()
{
    ... ..
    recurse();
    ... ..
}

int main()
{
    ... ..
    recurse();
    ... ..
}
```

recursive call

```
int main() {
    ... ..
    result = sum(number);
    ... ..
}

int sum(int n)
{
    if(n!=0)
        return n + sum(n-1);
    else
        return n;
}

int sum(int n)
{
    if(n!=0)
        return n + sum(n-1);
    else
        return;
}

int sum(int n)
{
    if(n!=0)
        return n + sum(n-1);
    else
        return n;
}

int sum(int n)
{
    if(n!=0)
        return n + sum(n-1);
    else
        return n;
}
```

3
3+3 = 6
is returned

2
1+2 = 3
is returned

1
0+1 = 1
is returned

0
0
is returned

ประเภทของการเวียนบังเกิด

- Direct recursion

- A function is said to be direct recursive if it calls itself directly

- Indirect recursion

- A function is said to be indirect recursive if it calls another function and this new function calls the first calling function again

```
int func1 (int n) {  
    if (n==1)  
        return 1;  
    else  
        return (n*func1(n-1));  
}
```

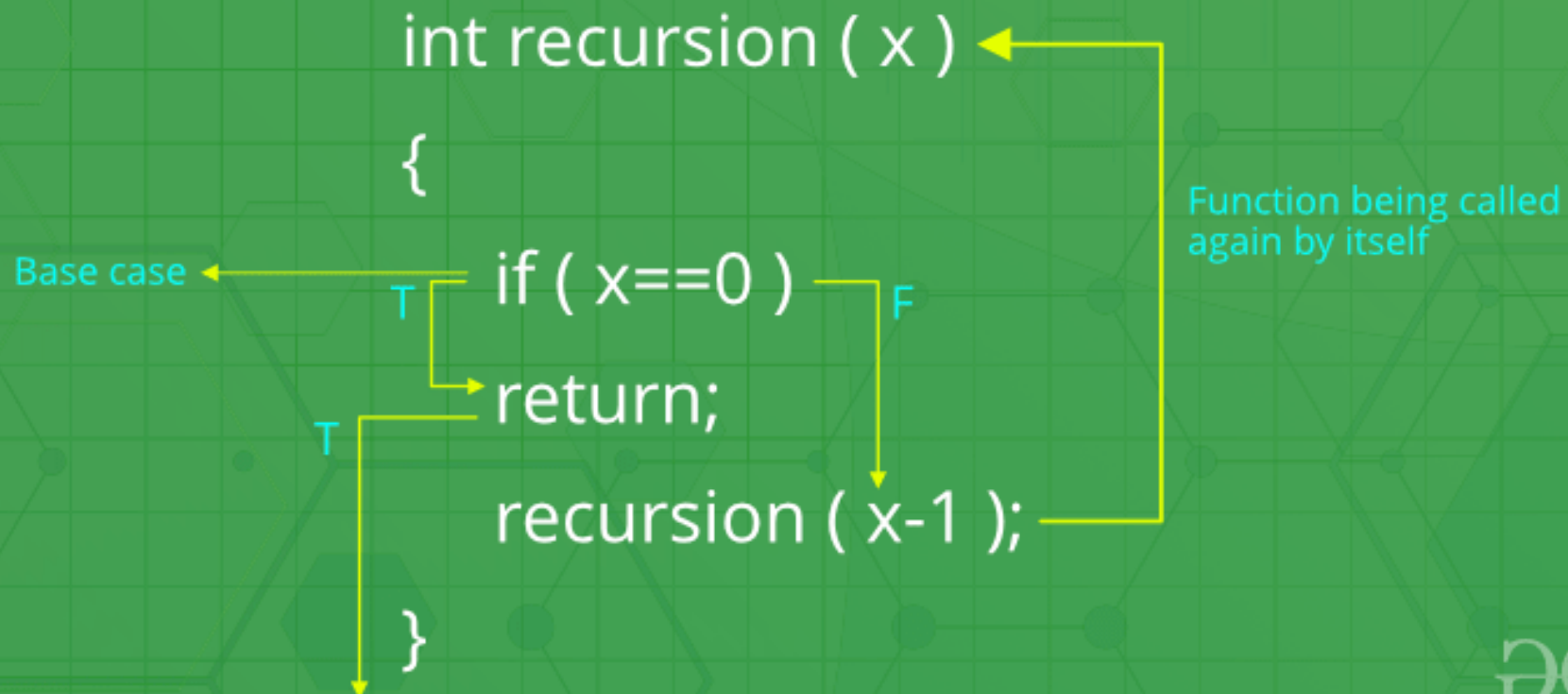
```
int func1(int n) {  
    if (n<=1) return 1;  
    else     return func2(n);  
}  
  
int func2(int n) {  
    return func1(n);  
}
```

Recursive Functions

```
int recursion ( x )  
{  
    if ( x==0 )  
        return;  
    recursion ( x-1 );  
}
```

Base case

Function being called again by itself

A diagram illustrating the logic of a recursive function. It shows the code for a function named 'recursion' that takes an integer 'x' as input. The function has a base case where if 'x' is equal to 0, it returns. If 'x' is not 0, it calls itself with 'x-1'. A yellow arrow points from the 'recursion (x-1);' line back to the function definition, labeled 'Function being called again by itself'. Another yellow arrow points from the 'if (x==0)' line to the text 'Base case'. There are also small 'T' and 'F' labels near the if statement.

- เมื่อมีกรณีอย่างน้อย 1 กรณีที่สามารถหาคำตอบได้ทันที (base case หรือ simple case)
- ในขณะที่กรณีอื่นๆสามารถกำหนดได้ใหม่ (redefine) ในรูปที่ง่ายขึ้นกว่าเดิม เรียกว่า recursive case
- ปัญหาสามารถลดรูปลงมาสู่ simple case
 - *If this is a simple case*
solve it
 - else*
redefine the problem using recursion

Terminating Condition

- **recursive functions** จะต้องมึเงื่อนไขหยุดการทำงานอย่างน้อย 1 เงื่อนไขเสมอ
 - โดยปกติ จะเป็นเงื่อนไขของกรณีพื้นฐาน **simple case**
- หากไม่มี **terminating condition** ฟังก์ชันอาจทำงานไม่หยุดเลย
 - เช่น ฟังก์ชันการคูณ if statement “if (n == 1) ...” คือ **terminating condition**.



เงื่อนไขในการหยุดการเวียนบังเกิด (Terminating condition)

- recursive functions จะต้องมียุทธการการทำงานอย่างน้อย 1 ยุทธการเสมอ
 - โดยปกติ จะเป็นยุทธการของกรณีพื้นฐาน base case
- หากไม่มี terminating condition ฟังก์ชันอาจทำงานไม่หยุดเลย
 - เช่น ฟังก์ชันการคูณ if statement “if (n == 1) ...” คือ terminating condition.

ตัวอย่าง

```
#include <stdio.h>
int sum(int n);

int main() {
    int number, result;

    printf("Enter a positive integer: ");
    scanf("%d", &number);
    result = sum(number);
    printf("sum = %d", result);
    return 0;
}

int sum(int num) {
    if (num!=0)
        return num + sum(num-1); // sum() function calls itself
    else
        return num;
}
```

```
#include <stdio.h>

long int fact( int n ) {
    if ( n <= 1 ) //base case
        return 1;
    else //recursive step
        return ( n * fact (n-1) );
} //end factorial

int main () {
    int i;
    for ( i = 1; i <=3; i++ )
        printf("%d! = %d\n",i, fact(i) );
    return 0;
}
```

Recursive Function: นับจำนวนตัวอักษรที่ต้องการใน string

- We can count the number of occurrences of a given character in a string.
 - e.g., the number of 's' in "Mississippi" is 4.

```
1.  /*
2.   * Count the number of occurrences of character ch in string str
3.   */
4.  int
5.  count(char ch, const char *str)
6.  {
7.
8.      int ans;
9.
10.     if (str[0] == '\0')                /* simple case */
11.         ans = 0;
12.     else                                /* redefine problem using recursion */
13.         if (ch == str[0])               /* first character must be counted */
14.             ans = 1 + count(ch, &str[1]);
15.         else                             /* first character is not counted */
16.             ans = count(ch, &str[1]);
17.
18.     return (ans);
19. }
```

A Recursive Function: Reverses Input Words (1/2)

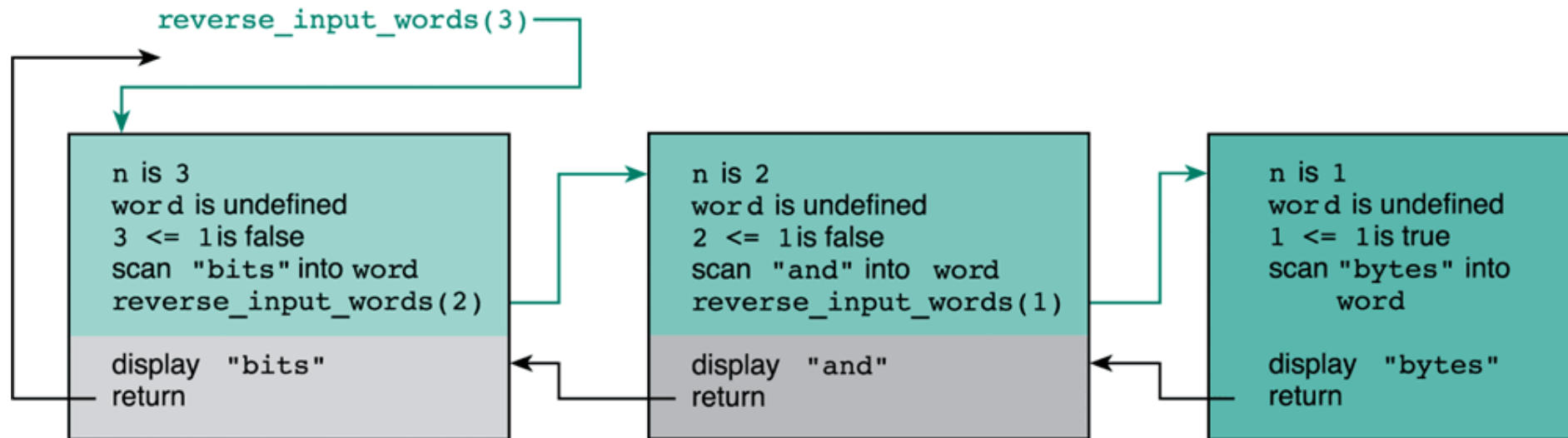
- The recursive concept can be used to reverse an input string.
 - It can also be done without recursion.

```
1.  /*
2.   *   Take n words as input and print them in reverse order on separate lines.
3.   *   Pre: n > 0
4.   */
5.  void
6.  reverse_input_words(int n)
7.  {
8.      char word[WORDSIZ]; /* local variable for storing one word */
9.
10.     if (n <= 1) { /* simple case: just one word to get and print */
11.
12.         scanf("%s", word);
13.         printf("%s\n", word);
14.
15.     } else { /* get this word; get the rest of the words in
16.              reverse order; then print the word */
17.
18.         scanf("%s", word);
19.         reverse_input_words(n - 1);
20.         printf("%s\n", word);
21.     }
22. }
```

The scanned word will not be printed until the recursion

The first scanned word is last printed.

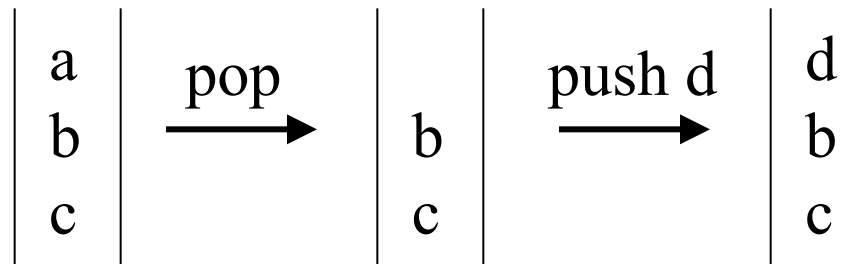
A Recursive Function that Reverses Input Words (2/2)



- Note that the recursive function is just an alternative solution to a problem.
 - You can always solve the problem without recursion.

How C Maintains the Recursive Steps

- ภาษา C เก็บรักษาค่าตัวแปรโดยใช้ **stack**
 - stack โครงสร้างข้อมูลที่จัดการข้อมูลในรูปแบบ LIFO
 - 2 operations คือ push และ pop



How C Maintains the Recursive Steps

- ทุกครั้งที่มีการเรียกฟังก์ชัน สถานะปัจจุบันของผู้เรียก (caller function) ได้แก่ parameters, local variables, and memory address เป็นต้น จะถูก push ลงใน stack
- เมื่อฟังก์ชันที่ถูกเรียกประมวลผลเสร็จ ค่าสถานะต่าง ๆ ของ caller function จะถูกนำกลับมาโดยการ pop จาก stack.
- This is sufficient to maintain the execution of the recursive function.
 - The execution state of each recursive step are stored and kept in order in the stack.

How to Trace Recursive Functions

- การ trace และ debug ฟังก์ชัน recursive ไม่ง่าย !!!
 - เช่น ถ้ามีการเรียกตัวเองนับร้อยครั้ง การ set break point หรือการ trace แบบ step-by-step จะน่าเหนื่อยมาก
- วิธีการที่นิยมคือ การเพิ่มคำสั่ง print เข้าไปเพื่อดูผลของการทำงานในแต่ละ recursive steps.

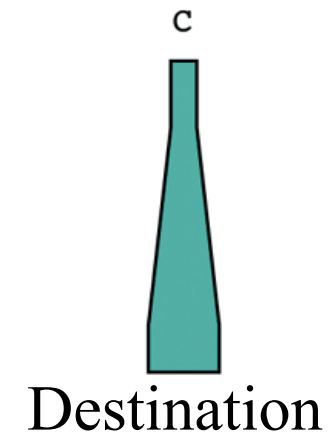
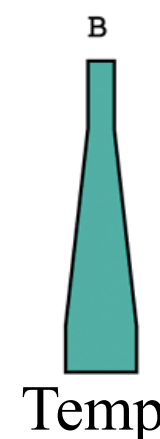
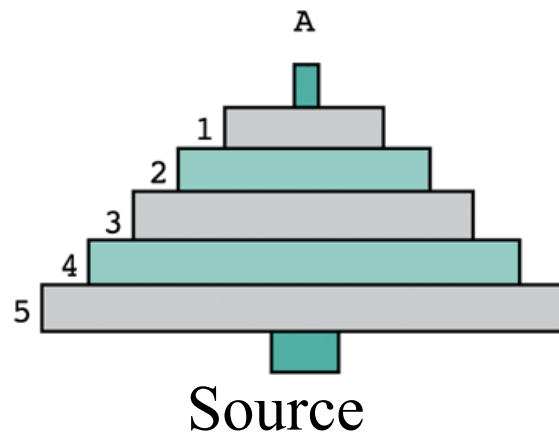
```
1.  /*
2.   * *** Includes calls to printf to trace execution ***
3.   * Performs integer multiplication using + operator.
4.   * Pre:  m and n are defined and n > 0
5.   * Post: returns m * n
6.   */
7.  int
8.  multiply(int m, int n)
9.  {
10.     int ans;
11.
12.     printf("Entering multiply with m = %d, n = %d\n", m, n);
13.
14.     if (n == 1)
15.         ans = m;      /* simple case */
16.     else
17.         ans = m + multiply(m, n - 1); /* recursive step */
```

Watch the input arguments passed into each recursive step

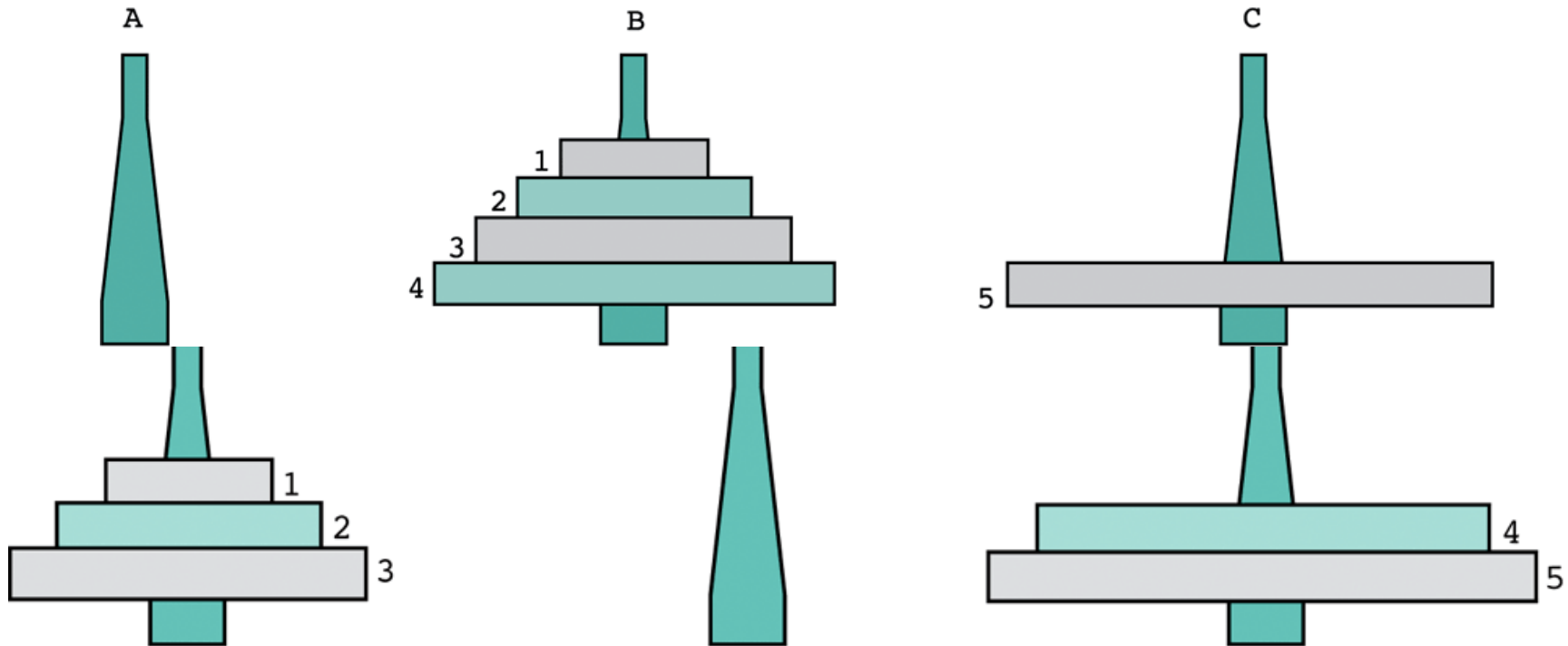
(continued)

หอคอยแห่งฮานอย (Tower of Hanoi) (1/3)

- เชื่อว่าเป็นการเล่นอย่างหนึ่งของพระสงฆ์ในเมืองฮานอย ประเทศเวียดนาม
- จุดประสงค์ของการเล่นนี้คือ ย้ายแผ่นจานทั้งหมด n จานจากเสาต้นทาง (source) ไปยังเสาปลายทาง (destination) โดยจะมีเสาทดให้ 1 เสา
- แผ่นจานแต่ละแผ่นจะมีขนาดต่างกัน
- กฎของการเล่นนี้คือ
 - ย้ายแผ่นจานได้ทีละ 1 แผ่น
 - ห้ามให้แผ่นจานขนาดใหญ่กว่าทับแผ่นจานขนาดเล็กกว่า
 - สมมติให้มี 3 เสา



หอคอยแห่งฮานอย (Tower of Hanoi) (2/3)



หอคอยแห่งฮานอย (Tower of Hanoi) (3/3)

- ปัญหานี้สามารถแก้ได้ง่ายเมื่อใช้ recursive

- Algorithm:

if n is 1 then

 move disk 1 from the source tower to the destination tower

else

1. move $n-1$ disks from the source tower to the temp tower.
2. move disk n from the source tower to the destination tower.
3. move $n-1$ disks from the temp tower to the source tower.

Advantages

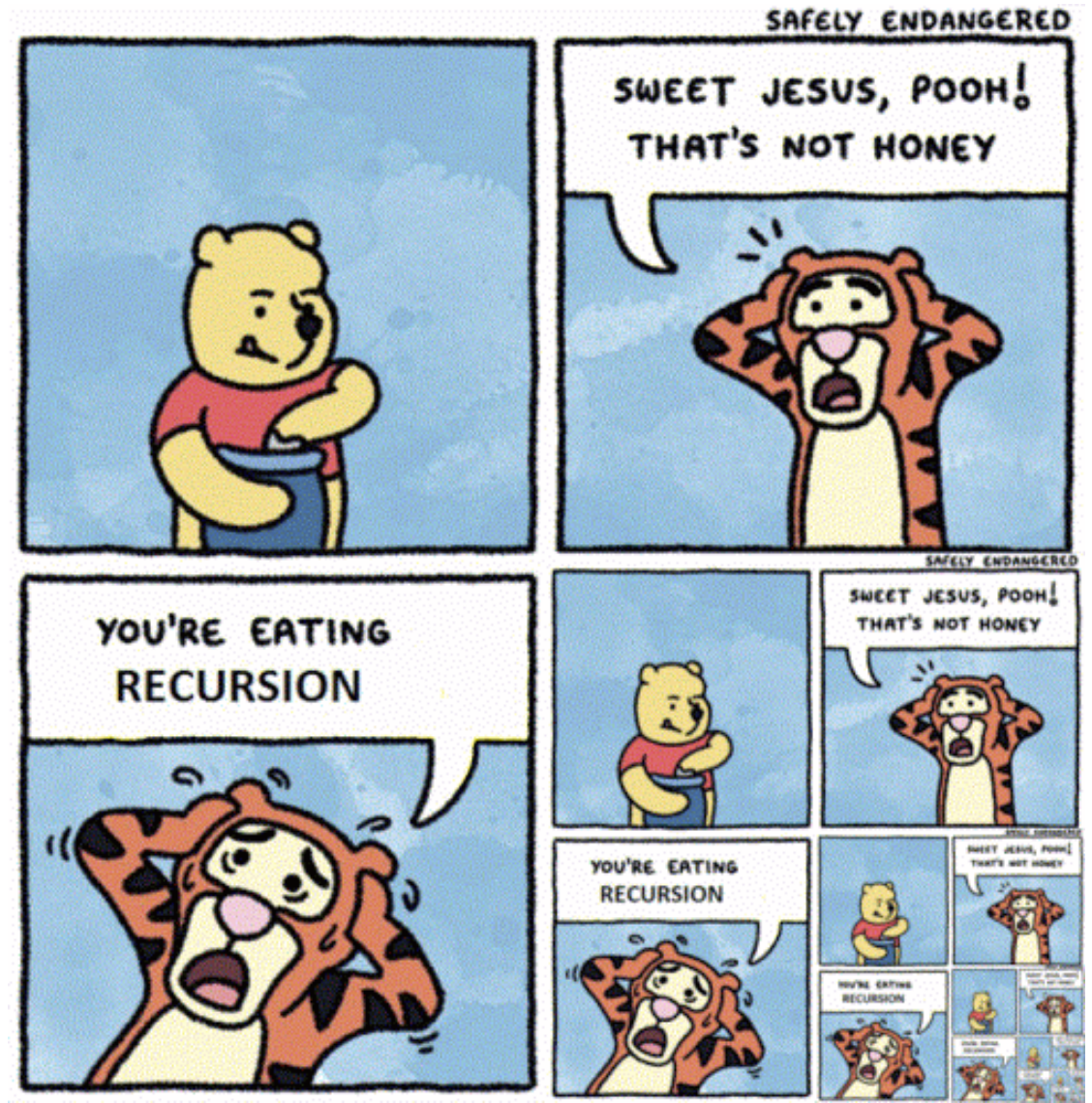
- Can complete a function with short lines of code
- Some problems can be easily solved
- Makes a program more readable
- Makes programs more neat
- Similar to repetitions

Disadvantages

- Needs more memory
- Slower than loops

Cautions!!!

- Forgetting base case can lead to infinite recursion
- Incorrect recursive step can lead to infinite recursion
- Repetitions terminates when loop's condition is false, but Recursion terminates when base case's is true



สรุปเรื่องการเวียนเกิด

- โปรแกรมเข้าใจง่าย
- โปรแกรมมีขนาดสั้น จำนวนบรรทัดน้อย
- สามารถแก้ปัญหาบางประเภทได้อย่างรวดเร็ว
- มีลักษณะคล้ายวงวน (Repetition)
- หยุดทำงานเมื่อเงื่อนไขเป็นจริง
- ใช้หน่วยความจำเยอะ
- ทำงานช้ากว่าวงวน



- จงเขียนโปรแกรมแบบ recursive สำหรับปัญหาแต่ละข้อต่อไปนี้
 - คำนวณค่าของ $n!$
 - คำนวณค่าของ a^n
 - หาตัวหารร่วมมากของจำนวนเต็ม a, b, c
 - A Recursive Linear Search Algorithm
 - A Recursive Binary Search Algorithm
 - A Recursive Algorithm for Fibonacci Numbers
 - An Iterative Algorithm for Computing Fibonacci Number